סדר עבודה ל־React: כיף Click

מבנים קודם את מה שכל השאר נשען עליו (התחברות וגישה לשרת), אחר כך את מסכי ההכנה, ורק בסוף את מסך המשחק החי, שהוא המורכב ביותר. כל שלב נבדק מול השרת לפני שעוברים לבא.

בכל שלב כדאי לפתוח את `FRONTEND-API-GUIDE.md`. כתוב שם אילו קריאות צריך לכל מסך, ואיך נראית התשובה.

שלב 0: הכנה

- ☐ להריץ את השרת במחשב (`npm run dev`) ולוודא ש־`/api-docs` נפתח
- ☐ ליצור פרויקט React + TypeScript (למשל עם Vite)
- ☐ לשים את כתובת השרת במשתנה סביבה `VITE_API_URL`
- ☐ להתקין React Query, React Hook Form, React Router, ו־Context או Redux Toolkit

בדיקה: הפרויקט עולה ומציג דף ריק.

שלב 1: שכבת התקשורת והטיפוסים

- ☐ טיפוסי TypeScript לפי האפיון: `User`, `Game`, `Question`, `Participant`, `LiveState`, `ApiError`
- ☐ פונקציה אחת לקריאה לשרת, שמוסיפה את `Authorization: Bearer` אוטומטית
- ☐ החזרת `error.message` כשיש שגיאה
- ☐ מעבר למסך התחברות כשחוזר 401

בדיקה: קריאה ל־`/api/health` מציגה `ok`.

שלב 2: התחברות והרשמה

- ☐ מסך התחברות (`POST /api/auth/login`)
- ☐ מסך הרשמה למנחה (`POST /api/auth/register`)
- ☐ שמירת ה־Token, ובדיקת `GET /api/auth/me` אחרי רענון של הדף
- ☐ Context או Redux שמחזיק את המשתמש המחובר ואת התפקיד שלו
- ☐ הצגת שגיאות השדות מ־`details` ליד כל שדה בטופס

בדיקה: התחברות עם `rachel.levi@kefclick.local` עובדת, והמנחה החסומה מקבלת הודעת חסימה.

שלב 3: ניתוב והרשאות

- ☐ נתיבים מוגנים: בלי התחברות מגיעים למסך התחברות
- ☐ מנחה מגיע ל"המשחקים שלי", ומנהל מגיע למסכי הניהול
- ☐ תפריט עליון עם שם המשתמש וכפתור יציאה

בדיקה: מנחה שמקליד כתובת של מסך ניהול לא נכנס אליו.

שלב 4: המשחקים שלי

- ☐ רשימה עם חיפוש, סינון לפי סטטוס ומיון (`GET /api/games`)
- ☐ יצירת משחק (`POST /api/games`)
- ☐ שכפול ומחיקה
- ☐ רענון הרשימה אחרי כל פעולה

בדיקה: מחפשים "גיאוגרפיה" ומסננים לפי `SETUP`.

שלב 5: עריכת משחק

- ☐ פרטי משחק: שם ותיאור
- ☐ משתתפים: טופס שם וטלפון, ורשימה עם עריכה ומחיקה
- ☐ שאלות: בחירת סוג (תחרותית או סקר), 4 תשובות ובחירת התשובה הנכונה
- ☐ בסקר, בחירת התשובה הנכונה מוסתרת
- ☐ שינוי סדר השאלות (`PUT /questions/order`)
- ☐ נטרול כל העריכה כשהסטטוס הוא לובי, רץ או מושהה

בדיקה: יוצרים משחק קטן עם 2 שאלות ומשתתף אחד עם הטלפון שלך.

שלב 6: לובי

- ☐ כפתור "פתח לובי" (`POST /lobby`)
- ☐ רענון של `/live` כל שנייה (למשל `refetchInterval` של React Query)
- ☐ רשימת משתתפים, עם סימון מי מחובר
- ☐ כפתורי "התחל" ו"בטל"

בדיקה בלי טלפון: ב־Swagger מריצים `GET /api/yemot/call` עם הטלפון של המשתתף, והשם שלו נצבע אצלך כמחובר.

שלב 7: מסך המשחק החי (הכי מורכב)

מסך אחד, שמשנה תצוגה לפי `phase`. כדאי לבנות בסדר הזה:

- ☐ `QUESTION`: שאלה, 4 תשובות וטיימר לפי `timeLeftMs`
- ☐ 5 המהירים בצד, מתוך `fastest`
- ☐ שמות קופצים: שומרים את `lastAnswerId` ושולחים אותו כ־`afterAnswerId`. כל שם חדש ב־`recentAnswers` מוצג שנייה אחת
- ☐ `REVEAL`: בשאלה תחרותית התשובה הנכונה וטבלת הניקוד, בסקר התפלגות התשובות
- ☐ השהיה, המשך וסיום, ותצוגת "מושהה" כש־`paused: true`
- ☐ `FINISHED`: 3 המנצחים, טבלה מלאה, וכפתור "שחק שוב"

בדיקה: משחק שלם דרך Swagger: מדמים הקשות עם `answer` מ־1 עד 4, וצופים בכל המעברים.

שלב 8: מסכי המנהל

- ☐ ניהול מנחים: רשימה, חיפוש, סינון חסומים
- ☐ הוספה, עריכה, חסימה ושחרור של מנחים
- ☐ כל המשחקים: סינון לפי מנחה וסטטוס, צפייה ומחיקה

שלב 9: ליטוש ובדיקה אמיתית

- ☐ הודעות טעינה ושגיאה בכל מסך
- ☐ עיצוב מימין לשמאל
- ☐ טקסט גדול במסך המשחק, כי הוא מוקרן לכיתה
- ☐ בדיקה עם טלפון אמיתי דרך ימות המשיח
- ☐ בדיקה עם כיתה